

RepoTracker — Strategy Notes

Open Source · Monetisation · Reach · Revenue · Ethics

1. The Core Tension

You want **open source + local-first** (for trust and reach) but also **paid features** (for revenue). These two goals pull in opposite directions — and that's not unique to you. It's one of the most debated problems in indie software.

2. The Cracking Problem — Realistic Assessment

Harsh truth: If your code is open source, paid features *will* get cracked. Not maybe. *Will*. Someone will fork it, remove the license check, and redistribute it. This happens to projects 10x more resourced than yours.

Your current approach — LemonSqueezy license validation stored locally after one online check — is **not crack-resistant** in an open source codebase. Anyone can read `license.js`, understand the validation flow, and patch it out. The obfuscation/binary route (like your v1.0 Tauri plan) helps more, but still isn't bulletproof.

So the real question isn't 'how do I prevent cracking' — it's 'how do I make cracking not matter much.'

3. Business Model Options

Option 1: Keep Current Model (Source-Available, Not True Open Source)

Your README says 'source-available' — that's actually the correct term. It's *not* open source by OSI definition if commercial use requires payment.

	Notes
Pros	Simple. You control it. Revenue flows naturally.
Cons	The community won't treat it as truly open source. Trust gap exists. Contributions are limited.
Verdict	Fine for now. But be honest — call it source-available, not open source.

Option 2: True Open Source + Hosted/Managed Service (Open Core SaaS)

Make everything open source. Charge for a **hosted, managed version** — cloud sync, a hosted team dashboard, priority support.

	Notes
Pros	Maximum reach. Community trust. Nobody "cracks" a self-hosted app.
Cons	Requires infrastructure. Doesn't fit your local-first philosophy well.
Verdict	Doesn't align with your vision.

Option 3: Open Core (What You're Essentially Doing Now)

Free core forever. Paid features behind a server-validated gate.

	Notes
Pros	Proven model (VS Code, GitLens, Posthog). Reach from free tier. Revenue from paying users.
Cons	The 'who cracks it' crowd is almost never your real customer.
Verdict	This is the right model for you. The cracking anxiety may be larger than the actual revenue loss.

Option 4: Sponsorware

Keep everything locked until you hit a GitHub Sponsors goal. Once hit, unlock publicly.

	Notes
Pros	Community-driven. Transparent. Creates urgency.
Cons	Slower. Missed the window — you already have v0.2 public.
Verdict	Missed the window for this approach.

4. What Actually Drives Revenue At Your Stage

At v0.2 with no significant install base yet, **reach matters infinitely more than lock-in**. The math is simple:

Scenario	Calculation	Revenue
100 users, 10% pay	$10 \times \$29$	\$290
10,000 users, 2% pay	$200 \times \$29$	\$5,800

Cracking reduces your conversion rate slightly. **A small audience is what kills your revenue**. Focus energy on distribution, not DRM.

5. Practical Steps to Increase Reach

Hacker News: Post "Show HN: RepoTracker — local Git mission control dashboard." One good HN thread = 500–2,000 GitHub stars overnight.

Reddit: r/programming, r/webdev, r/devops, r/selfhosted. The self-hosted community loves local-first tools.

Dev.to / Hashnode: "I built a local Git dashboard because I was tired of juggling 20 repos." Personal story + product = reach.

GitHub itself: Well-written README (yours is already solid), good topics/tags, and activity drives organic discovery.

Product Hunt: Plan it, don't rush it. A bad PH launch wastes your one shot.

X / Twitter: Short demo GIFs of AI commit message, Shelby terminal, health score get shared fast.

6. On Ethics

- Don't call it open source unless it is. Your license section already says 'source-available' — stick to that language everywhere.
 - Cracking your own app is a grey area for users who believe software freedom is a right. Fighting them publicly damages your reputation more than the lost \$29.
 - The people who crack it were likely never going to pay. Your real customer is a developer who values their time and will pay \$29 once without thinking twice.
 - If you move toward Tauri + compiled binaries, the ethical bar stays the same but the technical barrier to cracking rises. That's a legitimate choice.
-

7. One-Time Payment — The Revenue Problem

A one-time payment model **does not generate sustainable recurring revenue**. Your revenue curve looks like: Launch spike → steady trickle → slow decline. Unless you keep acquiring new users continuously, it dries up.

Option A: Keep One-Time, But Ship New Major Versions

RepoTracker v1.0 = new paid upgrade. Works if each version has genuinely new value. Downside: developers hate forced upgrades.

Option B: Switch to Annual Subscription

\$29 one-time = \$29 lifetime. One subscriber at \$5/month = \$60/year. The math compounds fast:

Subscribers	Monthly	Annual
100	\$500	\$6,000
500	\$2,500	\$30,000
1,000	\$5,000	\$60,000

Only works if you consistently keep shipping features — otherwise churn kills you.

Option C: Add a 'Pro Cloud' Layer Later (Recommended)

Local stays free. Optional cloud features on subscription: cloud-synced config across machines, hosted team dashboard, AI usage credits included monthly. Nobody feels forced. **This is the most ethical and sustainable path for your vision.**

Option D: Consulting / Enterprise Services

If RepoTracker gets real traction, companies will want custom deployment, enterprise licensing, priority support contracts. One enterprise deal is worth 100 individual sales.

Option E: GitHub Sponsors / Donations

Underrated for local-first tools. If users genuinely love it, some will sponsor you monthly. Won't make you rich, but covers costs + motivation.

8. Final Verdict

Priority	Action
Now	Focus 80% energy on reach, 20% on the product. Revenue follows audience.
Protect revenue	Accept that some cracking will happen. Your real customers won't crack it.
Terminology	"source-available" not "open source". Honest and still trustworthy.
Business model	Your current open-core, one-time payment model is right. Don't overcomplicate it.
Long-term	Tauri binary (v1.0) is your best technical moat. Relaunch with a subscription tier.
Parallel	Add optional cloud features that make subscription feel like a service, not a paywall.

The project looks genuinely good. Ship the reach strategy before you perfect the paywall.

The sequence is: Reach → Trust → Audience → Revenue. Not the other way around.